

A Réutilisation (5 points)

1.a)

Le listing 3 illustre la réutilisabilité du framework : une variable de type abstrait `Forme` peut référencer n'importe quelle sous-classe concrète (`Cercle`, `Ligne`). L'appel `f.drawDescription()` exploite le **polymorphisme** : le comportement dépend du type dynamique de l'objet. Cela permet d'étendre le framework sans modifier le code client (principe **Open/Closed**).

1.b)

`drawDescription()` de `Forme` est une fonction d'ordre supérieur car elle délègue une partie de son comportement à la méthode abstraite `getDescription()`, qui sera fournie par les sous-classes. Elle compose son exécution avec une fonction (méthode) fournie par le type concret (template method pattern).

1.c)

Type de paramétrage : **paramétrage par sous-typage** (inheritance-based, hot spot du framework).

Paramètre : la méthode abstraite `getDescription()` (point d'extension).

Argument : lors de l'exécution du listing 3, l'argument est la redéfinition concrète de `getDescription()` dans `Cercle` ou `Ligne`.

Dans `Dessin`, la boucle `for` utilise le même mécanisme : elle appelle `drawDescription()` polymorphiquement sur chaque élément.

2. Classe `Ligne`

```
public class Ligne extends Forme {
    private Point debut;
    private Point fin;

    public Ligne(Point debut, Point fin) {
        this.debut = debut;
        this.fin = fin;
    }

    @Override
    public String getDescription() {
        return "- une ligne de " + debut + " à " + fin;
    }
}
```

(Seules les parties utiles sont données.)

3. Organisation des tests

Pour un framework comme FramPict :

- Tests unitaires (JUnit) sur chaque sous-classe de `Forme` (vérifier `getDescription()`).
- Tests d'intégration sur `Dessin` : ajout de formes, ordre d'affichage, unicité via `Set` .
- Tests de non-régression lors d'extensions (nouvelles formes).
- Utiliser des fixtures pour créer des dessins complexes.
- Séparer tests du framework de base et tests d'extensions possibles par les utilisateurs.

B Réutilisation et typage statique (6 points)

1. Redéfinition de `equals` dans `Ligne`

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Ligne)) return false;
    Ligne other = (Ligne) obj;
    return debut.equals(other.debut) && fin.equals(other.fin) ||
        debut.equals(other.fin) && fin.equals(other.debut); // ligne non orientée
}
```

(Note : on accepte souvent la non-orientation pour une ligne.)

2. Commentaire avec principe de Liskov

La redéfinition respecte le **principe de substitution de Liskov** :

- Précondition non renforcée (accepte tout `Object`).
- Postcondition non affaiblie : conserve la sémantique d'équivalence (reflexive, symmetric, transitive).
- Comparaison basée sur les valeurs (début/fin), cohérente avec l'égalité logique d'une ligne.

3. Appel `f1.drawDescription()`

1. `drawDescription()` définie dans `Forme`
2. `getDescription()` définie dans `Ligne` (appel dynamique via polymorphisme)

4. `Vector<Forme> v = new Vector<Circle>();`

Non accepté par le compilateur Java (génériques invariants).

`Vector<Circle>` n'est pas un sous-type de `Vector<Forme>` bien que `Circle` soit sous-type de `Forme` . Cela violerait la sécurité de type (on pourrait ajouter une `Ligne` via la référence `Vector<Forme>`).

5. Factoriser `equals` dans `Forme`

Impossible directement car `Forme` est abstraite et n'a pas les attributs communs.

Solution viable : définir `equals` dans chaque sous-classe (comme fait) ou utiliser la **composition** avec

une classe valeur commune, mais ce n'est pas naturel ici. Aucune factorisation simple sans violer Liskov ou introduire des casts dangereux.

C Dessins arborescents (4 points)

1. Modification : pattern **Composite**

- Faire de `Dessin` une sous-classe de `Forme` (un dessin est une forme).
- Changer `contenu` en `List<Forme>` (ou garder `Set`, mais `List` pour ordre).

UML simplifié :

```

Forme (abstract)
  ↑
  ├── Cercle
  ├── Ligne
  └── Dessin
      - title: String
      - contenu: List<Forme>
  
```

Code clé dans `Dessin` :

```

public class Dessin extends Forme {
    private List<Forme> contenu = new ArrayList<>();
    private String title;

    public void add(Forme f) { contenu.add(f); }

    @Override
    public String getDescription() {
        return "Un dessin de " + title + " obtenu par :";
    }

    @Override
    public void drawDescription() {
        System.out.println(getDescription());
        for (Forme f : contenu) {
            System.out.print("— ");
            f.drawDescription();
        }
    }
}
  
```

2. Amélioration pour `dessinsInclus()`

Ajouter une référence inverse : chaque `Forme` garde la liste de ses parents (`List<Dessin> parents`).

Dans `Dessin.add(Forme f)` : si `f` n'est pas un `Dessin` (ou selon choix), ajouter `this` à ses parents.

Méthode :

```
public Collection<Dessin> dessinsInclus() {
    return Collections.unmodifiableList(parents);
}
```

Autoriser ou non plusieurs parents selon le choix (ici possible).

Pattern Composite + référence inverse (back pointers).

D Adaptation (3 points)

1. Pourquoi impossible directement

`java.awt.Point` n'hérite pas de `Forme` → ne peut pas être ajouté à la collection `Set<Forme>` (typage statique).

2. Solution : pattern **Adapter** (classe adaptateur)

UML :

```
Forme (abstract)
  ↑
FormePoint → java.awt.Point (contient)
```

Code clé :

```
public class FormePoint extends Forme {
    private Point point;

    public FormePoint(int x, int y) {
        this.point = new Point(x, y);
    }

    public FormePoint(Point p) {
        this.point = p;
    }

    @Override
    public String getDescription() {
        return "— un point de coordonnées " + point.getX() + "@" + point.getY();
    }
}
```

```
    }
}
```

3. Problème de perte du receveur initial

Si `FormePoint` redéfinit d'autres méthodes (ex. `move(int dx, int dy)`) en déléguant à son `Point` interne, un appel via une référence `Forme` perd l'accès aux méthodes spécifiques de `Point` (ex. `getLocation()`) → le receveur initial (`Point`) n'est plus accessible directement via l'interface `Forme`.

E Undo (2 points)

Solution : pattern Command + Stack

Remplacer `HashSet` par `Vector<Forme>` ou `ArrayList<Forme>` pour conserver l'ordre.

Ajouter un attribut :

```
private Stack<Forme> undoStack = new Stack<>();
```

Dans `add` :

```
public void add(Forme f) {
    contenu.add(f);
    undoStack.push(f);
}
```

Méthode `undo` :

```
public void undo() {
    if (!undoStack.isEmpty()) {
        Forme last = undoStack.pop();
        contenu.remove(contenu.size() - 1); // si Vector
    }
}
```

Architecture : **Command** (chaque ajout est implicitement une commande stockée dans une pile).

Alternative : objets `Command` explicites, mais ici simple pile suffit.